Create Grid With Exactly One Path

Daily Question — October 27, 2026

Problem Statement

You are given two integers m and n, representing the number of rows and columns of a grid. Construct any m x n grid consisting only of '.' (free cells) and '#' (obstacles).

A valid path starts at (0, 0), ends at (m - 1, n - 1), visits only free cells, and moves only right or down. Return any grid with exactly one valid path.

Examples

1. Input: m = 2, n = 3. One valid output: ["..#", "#.."].

2. Input: m = 3, n = 3. One valid output: ["..#", "#..", "##."].

3. Input: m = 1, n = 4. Output: ["...."].

Constraints: 1 <= m, n <= 25.

Solution: Top Row and Last Column

Make every cell in the top row free. Make every cell in the last column free. Set all other cells to '#'. This creates a single L-shaped path; it does not need to match the example output.

Java Solution

```
import java.util.Arrays;

class Solution {
    public String[] createGrid(int m, int n) {
        String[] result = new String[m];

        for (int row = 0; row < m; row++) {
            char[] cells = new char[n];
            Arrays.fill(cells, '#');

            if (row == 0) {
                Arrays.fill(cells, '.');
            } else {
                cells[n - 1] = '.';
            }

            result[row] = new String(cells);
        }

        return result;
    }
}
```

Correctness

Existence: Move right across the top row, then down the last column. Every cell on this route is free, including both endpoints.

Uniqueness: Before the last column, every cell directly below the top row is blocked. Therefore, a valid path must move right until the last column. There, it must move down to the destination. No alternative route exists.

Edge Cases

When m = 1, the entire row is free. When n = 1, the entire column is free. When m = n = 1, the only cell is free and the zero-move path is valid. Each case has exactly one path.

Walkthrough: m = 3, n = 3

```
...
##.
##.
```

Output: ["...", "##.", "##."]. The unique path is (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2).

Complexity

Time: O(m * n). Auxiliary space: O(n) for a temporary row. Output space: O(m * n).